Colmap-Free 3D Gaussian Splatting and the Effect of the Depth Map

1st Yelin Cheon
Stonybrook University
yelin.cheon@stonybrook.edu

***Abstract*—Recently, 3D reconstruction techniques such as NeRF and 3D Gaussian Splatting have gained significant attention in the field of computer vision. However, traditional 3D reconstruction methods typically rely on Structure-from-Motion (SfM) to estimate camera poses, which is not only time-consuming but also prevents an end-to-end pipeline. In this project, I explore the approach proposed in the Colmap-Free 3D Gaussian Splatting paper [1], which eliminates the need for explicit camera poses. Leveraging this advantage, I investigate whether 3D Gaussian Splatting can be successfully applied to videos generated by AI models. Instead of initializing Gaussians with poses extracted via SfM, I use depth maps to initialize the Gaussians and examine how different depth estimation models affect the reconstruction quality. Specifically, I conduct comparative experiments using three state-of-the-art monocular depth estimation models to assess their impact on the performance of Colmap-Free 3DGS.**

***Index Terms*—Computer Graphics, Computer Vision, 3D Gaussian Splatting [2], Colmap Free 3DGS, Depth estimation**

I. Introduction

In recent years, 3D reconstruction techniques have attracted significant attention in the field of computer vision. Methods such as Neural Radiance Fields (NeRF) [3] and 3D Gaussian Splatting have shown impressive capabilities in generating high-quality 3D scenes from static images or videos. Most of these methods rely on accurate camera poses to recover the 3D structure of a scene, which are typically obtained through Structure-from-Motion (SfM). However, extracting camera poses using SfM is time-consuming and prevents the entire pipeline from being trained in an end-to-end fashion.

To address this limitation, recent work has proposed Colmap-Free 3D Gaussian Splatting, which enables the reconstruction of 3D scenes without requiring explicit camera poses. Instead, this approach utilizes depth maps to initialize the 3D Gaussians, offering greater efficiency and potential for end-to-end training.

In this project, I build upon the Colmap-Free 3DGS framework to test whether 3D Gaussian Splatting can be applied to videos generated by AI models, specifically using SORA, a recent generative video model. I also examine the impact of depth map quality on reconstruction performance by comparing results obtained using three different monocular depth estimation models: MiDaS [4], ZoeDepth [5], and Depth Anything [6].

II. Related Works

A. Depth Estimation

A depth map is a single-channel image in which each pixel represents the relative distance from the camera, enabling the geometric structure of a scene to be expressed in 2D image space. This depth information provides critical structural cues for a variety of visual perception tasks, including 3D reconstruction, scene understanding, and domain adaptation.

Recent studies have proposed methods that predict the initial 3D structure of a scene using depth maps without relying on Structure-from-Motion (SfM)-based camera pose estimation. A notable example is COLMAP-Free 3D Gaussian Splatting, which generates an initial set of Gaussians using monocular depth estimation without requiring explicit camera poses, and subsequently performs high-quality 3D reconstruction.

Monocular depth estimation refers to the task of predicting depth from a single image. Representative zero-shot monocular depth estimation models include MiDaS, ZoeDepth, and Depth Anything. These models enhance depth estimation performance by employing scale-invariant loss (MiDaS), leveraging large and diverse training datasets for better generalization (ZoeDepth), or adopting a teacher-student architecture for self-supervised learning to improve both depth accuracy and generalization (Depth Anything).

In this project, I conduct a comparative analysis of these three monocular depth estimation models and demonstrate that depth-based structural information can be effectively used for 3D scene reconstruction, while significantly reducing the complexity of SfM-based pipelines.

B. Novel View Synthesis

Novel View Synthesis (NVS) aims to generate images of a scene from unseen viewpoints, given a limited number of input images. It is a key application area in 3D reconstruction and neural rendering. While early methods relied on geometry-based approaches, recent advances have focused on learning-based techniques using neural representations.

A prominent example is Neural Radiance Fields (NeRF), which models a continuous volumetric scene representation by predicting color and density for any 3D point and viewing direction. NeRF achieves high-quality synthesis results but suffers from slow rendering speed and computationally

expensive optimization, making it less practical for real-time applications.

To overcome these limitations, 3D Gaussian Splatting (3DGS) was introduced as an alternative approach. Instead of relying on volumetric ray marching, 3DGS represents a scene using thousands of 3D Gaussians, each parameterized by its position, orientation, scale, color, and opacity. These Gaussians are directly projected and rasterized in image space, enabling high-quality and real-time novel view synthesis.

In this work, I follow the 3DGS-based approach but aim to eliminate the dependency on COLMAP by leveraging monocular depth maps to initialize the 3D Gaussians. This enables an efficient and scalable pipeline for novel view synthesis without requiring SfM-based camera pose estimation.

C. Text-to-Video Generation

Recent advancements in text-to-video generation have led to the emergence of powerful models capable of producing visually compelling and temporally coherent videos from a single text prompt. Among them, Sora by OpenAI stands out for its ability to generate high-resolution videos with strong physical consistency and scene composition, outperforming previous models in both fidelity and realism.

These models are typically trained on large-scale video datasets and employ architectures based on transformers for frame prediction or leverage 3D-aware latent spaces to maintain temporal continuity and visual coherence.

However, the videos generated by such models do not explicitly contain accurate 3D structural information, making it challenging to reconstruct detailed 3D scenes or perform precise view synthesis from novel camera poses.

In this project, I utilize videos generated by models such as Sora and apply the Colmap-Free 3D Gaussian Splatting method, which does not rely on camera pose estimation, to accurately reconstruct 3D scenes and directly synthesize novel views from the generated video.

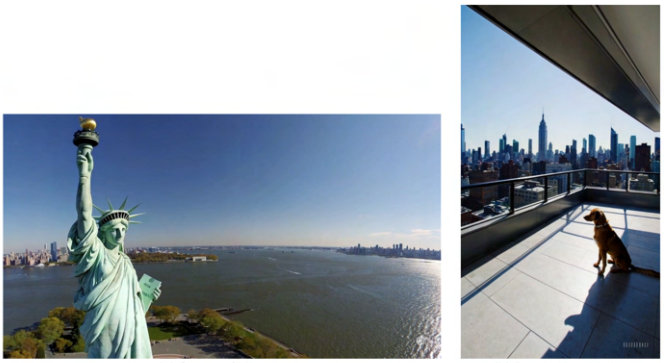
III. Method

In this project, I (1) generate videos using the video generative model Sora, (2) apply Colmap-Free 3D Gaussian Splatting to the generated videos, and (3) analyze and compare the impact of different depth estimation models.

A. Generate Video with SORA Model

To ensure high-quality 3D reconstruction from SORA-generated videos, I applied several prompt-level constraints, including control over camera motion, scene illumination, and object dynamics:

- **Scene Transition Speed:** Since the video is divided into individual frames and the model is trained on these sequentially, excessively fast scene transitions may result in inconsistent spatial correspondence across frames. This can negatively affect the reconstruction quality. Therefore, I encouraged smooth and gradual camera motion within the generated videos.
- **Image Quality and Illumination:** Low-quality or poorly lit frames can significantly degrade the performance of depth estimation models due to large domain gaps, especially in zero-shot scenarios. To mitigate this, I ensured that the generated videos featured high-resolution imagery and well-lit, daytime scenes.
- **Static Objects:** Object motion within a scene introduces inconsistencies that impair reconstruction quality and typically require additional techniques such as dynamic scene modeling or object tracking. To simplify the process and maintain static assumptions during Gaussian optimization, I constrained the prompts to generate scenes with mostly static objects.



(a) Statue of Liberty (b) Dog in New York

Fig. 1. (a) and (b) are sample frames extracted from videos generated using the SORA model.

The prompts were written according to the constraints described above, as follows:

- **Statue of Liberty:** A 10-second video where the camera slowly orbits around the Statue of Liberty. The scene takes place during the daytime with high resolution and consistent lighting. The subject remains completely static, and the background may include trees, buildings, or mountains.
- **Dog in New York:** An 8-second video where the camera slowly orbits around the dog. The scene takes place during the daytime with high resolution and consistent lighting. The subject remains completely static, and the background is New York City.

B. Gaussian Initialization

In this project, camera poses are not available; therefore, I utilize a depth map to initialize the 3D Gaussians. The initialization process uses only the first frame of the generated video. Specifically, I input the first frame into a monocular depth estimation model to predict a depth map. This depth map is then projected into 3D space using the known camera intrinsics to generate a point cloud.

From this point cloud, I extract geometric and appearance properties such as the position, scale, and color of each point, which are then used to initialize the parameters of the 3D Gaussians. Since I used images extracted from a generated video, the exact camera intrinsics were not available.

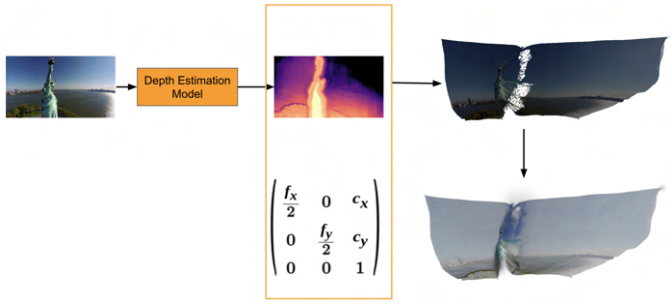


Fig. 2. Starting from the left, the first frame of the video is fed into the depth estimation model MiDaS to predict a depth map. Using this depth map and the camera intrinsics, a point cloud is generated (top right). This predicted point cloud is then used to initialize the Gaussians (bottom right).

Therefore, I empirically set the field of view (*FoV*) to 79.0 degrees. The intrinsic parameters were then estimated using the following procedure:

$$FoV = 79.0 \tag{1}$$

$$FoV_x = FoV \cdot \frac{\pi}{180} \tag{2}$$

$$f_x = f_y = \frac{\text{width}}{2 \cdot \tan\left(\frac{FoV_x}{2}\right)} \tag{3}$$

$$c_x = \frac{\text{width}}{2}, \quad c_y = \frac{\text{height}}{2} \tag{4}$$

Here, (f_x, f_y) denote the focal lengths, and (c_x, c_y) represent the principal point coordinates. The width and height correspond to the resolution of the video frames.

C. Refine Initialized Gaussian

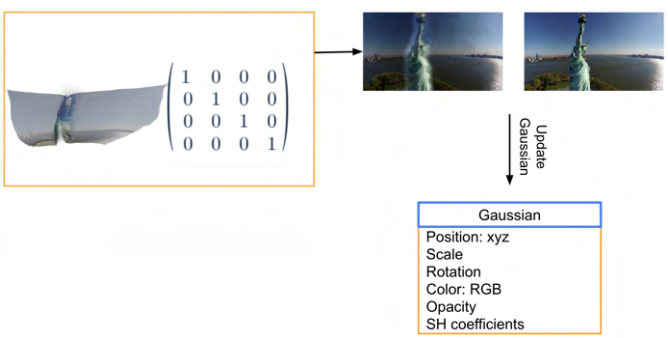


Fig. 3. This figure illustrates the process of refining the initialized Gaussians.

After initializing the Gaussians from the point cloud, I refine them to improve reconstruction quality. This refinement is necessary because the quality of the initial Gaussians significantly affects the overall fidelity of the reconstructed scene.

The initialized Gaussians are rendered into a 2D image using the initial camera pose. The resulting 2D image is then compared to the original first frame (i.e., the ground truth) to compute a photometric loss. This loss is used to update the parameters of each Gaussian component, including position, scale, color, SH coefficients and opacity. Since no initial camera pose is available, I initialize it using the identity transformation matrix (a 4×4 matrix with 1s on the diagonal). The initial Gaussians are then refined over 1000 iterations, with each component being updated using a tailored learning rate.

D. Local 3DGS for Pose Estimation

The local 3D Gaussian Splatting (3DGS) method is to estimate the relative camera pose between adjacent video frames. The key idea is to leverage the explicit 3D Gaussian representation to directly model camera motion as a rigid SE(3) transformation, without optimizing the Gaussians themselves.

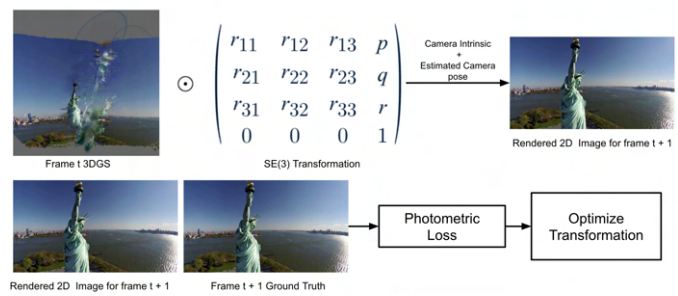


Fig. 4. Overview of pose estimation pipeline. A 3DGS generated from frame t is transformed via a learnable SE(3) operation to match the next frame. The transformed Gaussians are rendered into 2D, and compared to the ground-truth frame $t{+}1$ via photometric loss. The transformation is optimized accordingly.

Given a refined 3DGS representation G_t^* constructed from frame t, I estimate the relative camera pose to frame $t{+}1$ by applying a learnable SE(3) transformation T_t:

$$G_{t+1} = T_t \odot G_t^* \tag{5}$$

I then render a 2D image $\mathcal{R}(G_{t+1})$ using the transformed Gaussians and the known camera intrinsics. This rendered image is compared to the ground-truth next frame I_{t+1} using a photometric loss:

$$T_t^* = \arg\min_{T_t} \mathcal{L}_{\text{rgb}}(\mathcal{R}(T_t \odot G_t), I_{t+1}) \tag{6}$$

During this optimization, the Gaussians themselves are kept fixed, and only the SE(3) transformation is updated. This separation ensures that the optimized transformation purely captures the relative camera motion, not changes in scene structure. This local 3DGS process was trained for 500 iterations.

E. Global 3DGS

While local 3DGS effectively captures relative camera motion between adjacent frames, it lacks the ability to model the global structure of the scene. Since it only relies on short-term frame pairs, it may fail to enforce consistency over the entire video sequence. To address this limitation, the global 3DGS stage is needed that incorporates long-range dependencies.

In this stage, for each current frame, I randomly sample a previous frame $t-x$. The probability of sampling such a non-adjacent frame is set to 0.3 during the early training phase and gradually increases to 0.7 in the later stages. This scheduling strategy reflects the intuition that the model already captures good local representations in the early stage, and thus benefits more from long-range updates in later iterations.

After selecting frame $t-x$, I retrieve its estimated camera pose and render the current Gaussians G_t from that viewpoint. I then compute a photometric loss between the rendered image and the ground-truth image of frame $t-x$:

$$\mathcal{L}_{\text{rgb}} = \mathcal{L}_{\text{rgb}}(\mathcal{R}(G_t, P_{t-x}), I_{t-x}) \tag{7}$$

Finally, I jointly optimize both the camera pose of frame $t-x$ and the attributes of Gaussians G_t to minimize this loss. This enables the model to learn globally consistent scene geometry and camera trajectories across the full video.

In both the local and global 3DGS stages, the photometric loss $\mathcal{L}_{\text{rgb}}$ is computed as a weighted combination of the L_1 loss and the D-SSIM loss. This formulation balances pixel-wise differences and perceptual similarity between the rendered image and the ground truth.

The loss is defined as:

$$\mathcal{L}_{\text{rgb}} = (1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}} \tag{8}$$

I use $\lambda = 0.2$ in all experiments like original paper to give higher weight to the L_1 loss while still incorporating structural similarity from D-SSIM.

IV. Experiments

A. Implementation Details

In this section, I introduce the experimental details of my approach.

My implementation closely follows the original Colmap-Free 3D Gaussian Splatting (3DGS) framework [1]. I reuse its core pipeline while extending it in multiple directions to evaluate and enhance performance under different conditions.

Dataset and Preprocessing. I split generated videos into individual frames. I use videos generated by Sora, uniformly divided into 100 frames. Among these, 88 frames are used for training and 12 frames are reserved for testing. For evaluation using ground-truth camera poses, I additionally adopt the Francis scene from the Tanks and Temples dataset.

Depth Estimation Models. Unlike the original paper, which fixes MiDaS [4] as the depth estimator, I conduct a comparative study across three different monocular depth models: MiDaS, ZoeDepth [5], and Depth Anything [6], to assess their impact on Gaussian initialization quality.

Local 3DGS Training. To evaluate the effectiveness of the local 3DGS stage, I train the model for different iteration lengths (300 and 500 iterations) and analyze their impact. The local 3DGS is trained to estimate relative camera poses between adjacent frames, but does not capture global scene structure.

Global 3DGS Extension. To enforce long-range consistency, I incorporate a global 3DGS stage. At each step, a previous frame $t-x$ is randomly sampled with a probability of 0.3 during the early training phase and 0.7 in the later phase. The current Gaussians are rendered from the sampled frame's estimated camera view, and a photometric loss is computed with its ground truth. Both the camera pose and Gaussians are jointly optimized.

Opacity Resetting. Following the original paper, I continuously reset opacity values to allow newly observed frames to be integrated into the current Gaussian representation.

Optimization and Learning Rate Schedule. The camera pose is represented as a quaternion $q \in SO(3)$ and a translation vector $t \in \mathbb{R}^3$. I adopt a learning rate warm-up schedule, starting at 10^{-5} and gradually decaying to 10^{-6} as training progresses. All experiments were conducted on a laptop equipped with an RTX 4070 GPU.

Visualization. To facilitate inspection of reconstruction quality, I additionally implemented to visualize point clouds and rendered 3D views during training.

Evaluation Metrics. For novel view synthesis, I report PSNR, SSIM, and LPIPS scores. For camera pose estimation, I use ATE (Absolute Trajectory Error) and RPE (Relative Pose Error). Since Sora-generated videos lack ground-truth poses, I use the Francis scene in Tanks and Temples for pose evaluation.

B. Results

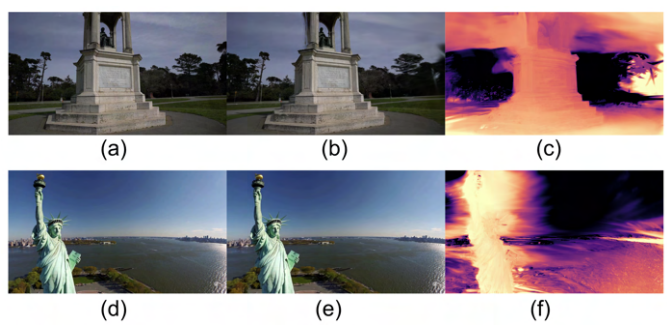


Fig. 5. (a), (b), and (c) show the novel view synthesis results on the Francis scene from the Tanks and Temples dataset, while (d), (e), and (f) correspond to the novel view synthesis results on the Statue of Liberty scene generated using Sora. From left to right, each triplet displays the ground truth image, the 2D rendered image via 3DGS (Novel view synthesis result), and the estimated depth map. The depth estimation model used in both cases is MiDaS.

Table I compares Colmap-Free 3DGS with other models that do not utilize camera poses. Overall, Colmap-Free 3DGS demonstrates the best performance across most scenes. The newly added scenes, Statue of Liberty and Dog in New York, are generated using Sora and represent my own custom dataset. For fair comparison, I use the MiDaS depth estimation model and set the number of local 3DGS iterations to 500.

Compared to Dog in New York, the Statue of Liberty sequence features smoother camera motion and fewer fine details in the background. When comparing PSNR scores,

scenes	Colmap-Free 3DGS			Nope-NeRF [7]			BARF [8]			NeRFmm			SC-NeRF [9]		
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Church	30.23	0.93	0.11	25.17	0.73	0.39	23.17	0.62	0.52	21.64	0.58	0.54	21.96	0.60	0.53
Barn	31.23	0.90	0.10	26.35	0.69	0.44	25.28	0.64	0.48	23.21	0.61	0.53	23.26	0.62	0.51
Museum	29.91	0.91	0.11	26.77	0.76	0.35	23.58	0.61	0.55	22.37	0.61	0.53	24.94	0.69	0.45
Family	31.27	0.94	0.07	26.01	0.74	0.41	23.04	0.61	0.56	23.04	0.58	0.56	22.60	0.63	0.51
Horse	33.94	0.96	0.05	27.64	0.84	0.26	24.09	0.72	0.41	23.12	0.70	0.43	25.23	0.76	0.37
Ballroom	32.47	0.96	0.07	25.33	0.72	0.38	20.66	0.50	0.60	20.03	0.48	0.57	23.61	0.61	0.48
Francis	32.72	0.91	0.06	29.48	0.80	0.18	25.85	0.69	0.57	25.40	0.60	0.52	26.46	0.73	0.49
Ignatius	28.43	0.90	0.09	23.96	0.61	0.47	21.78	0.47	0.60	21.16	0.45	0.60	23.00	0.55	0.53
Statue of Liberty	**32.58**	**0.95**	**0.13**	–	–	–	–	–	–	–	–	–	–	–	–
Dog in New York	**20.46**	**0.79**	**0.27**	–	–	–	–	–	–	–	–	–	–	–	–

TABLE I

NOVEL VIEW SYNTHESIS COMPARISON USING PSNR↑, SSIM↑, AND LPIPS↓. COLMAP FREE 3DGS METHOD ACHIEVES CONSISTENTLY BETTER PERCEPTUAL QUALITY, ESPECIALLY IN NEW SCENES GENERATED BY SORA.

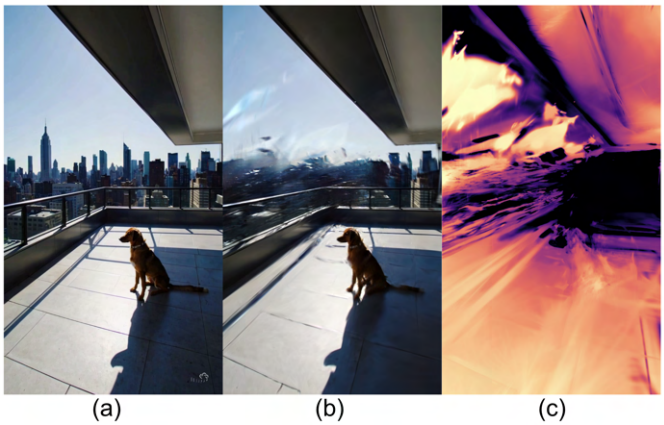


Fig. 6. (a), (b), and (c) show the novel view synthesis results on the dog in New York generated using Sora. From left to right, each triplet displays the ground truth image, the 2D rendered image via 3DGS (Novel view synthesis result), and the estimated depth map. The depth estimation model used in both cases is MiDaS.

Statue of Liberty achieves 32.58, while Dog in New York scores 20.46, suggesting that the quality of the generated video and the prompt-level constraints significantly impact the final reconstruction performance.

C. Ablation Study

1) Local 3DGS Iteration: To analyze the impact of Local 3DGS iterations, I conducted experiments using both 300 and 500 training steps. In addition to the Statue of Liberty scene, I included the Francis sequence from the Tanks and Temples dataset for fair comparison across different scene types.

As shown in Table II, the PSNR for the Statue of Liberty scene increased by approximately **0.49%**, while the Francis scene showed a larger improvement of **6.6%** when increasing the number of iterations from 300 to 500.

In terms of training time, using 88 frames, the 500-iteration setting required approximately **5 to 5.5 hours**, while the 300-iteration setup only took around **1.5 to 2 hours**. Although the 500-iteration training leads to slightly better performance, the marginal gains—especially for scenes with already clean structure—suggest that using 300 iterations offers a more efficient trade-off between quality and compute.

2) Results from different depth models: To evaluate the effect of different depth maps on 3D reconstruction quality, I compared three monocular depth estimation models: MiDaS, ZoeDepth, and Depth Anything. The evaluation was conducted on the *Dog in New York* video generated by Sora. As in the Statue of Liberty setting, the video was divided into 100 frames, with 88 used for training and 12 for testing.

Each depth model has distinct characteristics:

- **MiDaS (ResNet-101 / ViT)**: Predicts relative depth using a scale- and shift-invariant loss. Trained on a diverse set of datasets (ReDWeb, DIML, MegaDepth, 3D Movie, etc.), it is robust across a wide range of environments.
- **ZoeDepth (BEiT-Large)**: Supports both relative and metric depth prediction. Trained on 12 datasets including HRWSI, BlendedMVS, ReDWeb, and DIML-Indoor, with strong emphasis on generalization performance.
- **Depth Anything (ViT-L)**: Trained on a mix of labeled and self-supervised data. Uses a feature alignment loss to preserve strong semantic representations from the encoder.

As shown in Table III, **ZoeDepth** achieved the best performance in PSNR, SSIM, and LPIPS. This suggests that, for synthetic data like Sora-generated videos, generalization ability plays a critical role in achieving high-quality depth predictions and subsequent 3D reconstructions.

3) Camera Pose using Different Depth Model: To assess the impact of different depth models on camera pose estimation, I evaluated MiDaS, ZoeDepth, and Depth Anything using the *Francis* scene from the Tanks and Temples dataset. Since Sora-generated videos do not provide ground-truth camera poses, this dataset was used for pose evaluation with known reference trajectories.

As shown in Table IV, the MiDaS model achieved the best overall performance in terms of both translational RPE and absolute trajectory error (ATE). This suggests that MiDaS provides more globally coherent depth estimations, which are particularly beneficial for initializing the Gaussian scene structure.

I hypothesize that accurate pose estimation benefits more from capturing the overall geometric structure of the scene than from high-frequency detail. Since pose prediction relies heavily on the alignment of 3D Gaussians initialized from depth, models like MiDaS that focus on global depth consistency perform better in this setting.

Local 3DGS Iteration	Statue of Liberty			Francis (Tanks)			Training Time
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	(88 frames)
300 Iter	32.427	0.952	**0.117**	30.792	0.881	0.173	**1.5–2 hrs**
500 Iter	**32.584**	**0.953**	0.127	**32.823**	**0.915**	**0.137**	5–5.5 hrs

TABLE II

COMPARISON OF LOCAL 3DGS PERFORMANCE UNDER DIFFERENT TRAINING ITERATIONS. PSNR/SSIM/LPIPS VALUES ARE REPORTED ALONG WITH APPROXIMATE TRAINING TIME USING 88 FRAMES.

Depth Estimation Model	PSNR ↑	SSIM ↑	LPIPS ↓
MiDaS	20.461	0.791	0.274
ZOE	**21.983**	**0.836**	**0.249**
Depth Anything	20.915	0.808	0.261

TABLE III

COMPARISON OF DIFFERENT DEPTH ESTIMATION MODELS ON THE *Dog in New York* SCENE. ZOEDEPTH OUTPERFORMS OTHERS ACROSS ALL METRICS.

Depth Estimation Model	RPE Trans ↓	RPE Rot ↓	ATE ↓
MiDaS	**0.029**	0.154	**0.006**
ZoeDepth	0.057	**0.087**	0.010
Depth Anything	0.043	0.136	0.009

TABLE IV

COMPARISON OF CAMERA POSE ESTIMATION ACCURACY ON THE *Francis* DATASET USING DIFFERENT DEPTH ESTIMATION MODELS.

V. CONCLUSION

The quality of the input video plays a significant role in the final reconstruction performance. This is clearly demonstrated in the PSNR comparison between the Dog in New York and Statue of Liberty scenes, where the latter—characterized by smoother camera motion and less background complexity—achieved considerably higher scores. This highlights how factors such as motion stability and visual clarity in generated videos directly affect reconstruction quality.

In terms of training efficiency, I found that reducing the number of local 3DGS iterations from 500 to 300 significantly decreases training time—from approximately 5–5.5 hours to just 1.5–2 hours—while still maintaining acceptable reconstruction quality. This suggests that, depending on the target application, a shorter training schedule may provide a favorable trade-off between performance and computational cost.

When evaluating the impact of different depth estimation models on Gaussian initialization, the ZoeDepth model consistently outperformed others across PSNR, SSIM, and LPIPS metrics. This demonstrates ZoeDepth's strong generalization capabilities, which are particularly valuable when working with synthetic or domain-shifted inputs, such as those generated by video diffusion models.

Finally, in the task of camera pose estimation, the MiDaS model achieved the best overall performance, especially in terms of translational RPE and absolute trajectory error (ATE). I attribute this to MiDaS's ability to generate globally consistent depth maps, which are critical for initializing accurate 3D Gaussians and aligning poses. These findings collectively emphasize the importance of careful component selection—including video quality, depth model, and training strategy—when optimizing 3D Gaussian Splatting pipelines for downstream tasks.

REFERENCES

[1] Y. Fu, S. Liu, A. Kulkarni, J. Kautz, A. A. Efros, and X. Wang, "Colmap-free 3d gaussian splatting," 2024.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," 2023.

[3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," 2020.

[4] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," 2020.

[5] S. F. Bhat, R. Birkl, D. Wofk, P. Wonka, and M. Müller, "Zoedepth: Zero-shot transfer by combining relative and metric depth," 2023.

[6] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," 2024.

[7] W. Bian, Z. Wang, K. Li, J.-W. Bian, and V. A. Prisacariu, "Nope-nerf: Optimising neural radiance field with no pose prior," 2023.

[8] C.-H. Lin, W.-C. Ma, A. Torralba, and S. Lucey, "Barf: Bundle-adjusting neural radiance fields," 2021.

[9] L. Song, G. Wang, J. Liu, Z. Fu, Y. Miao, and Hesheng, "Sc-nerf: Self-correcting neural radiance field with sparse views," 2023.